

Deep State Space Models: From Convolutions to S4, S5, and Mamba

Ethan Read

May 13, 2026

Contents

Preface	5
1 Convolutions For Sequences	6
1.1 Why Start With Convolutions?	6
1.2 Sequences, Channels, And Shapes	6
1.3 Convolution Versus Cross-Correlation	7
1.4 Causality	7
1.5 Padding And Boundary Effects	8
1.6 Stride And Downsampling	8
1.7 Receptive Field	9
1.8 Dilated Convolution	9
1.9 Channel Mixing	9
1.9.1 Pointwise Convolution	10
1.9.2 Depthwise Convolution	10
1.9.3 Grouped Convolution	10
1.10 Convolution As A Matrix	10
1.11 Finite Impulse Response Filters	11
1.12 Convolution And Parallelism	11
1.13 FFT Convolution	11
1.14 Convolutional Sequence Blocks	12
1.15 CNNs, RNNs, And Transformers	12
1.16 The Bridge To State Space Models	13
1.17 What To Remember	13
1.18 Exercises	13
2 Linear Systems And Filters	15
2.1 Why Linear Systems Matter	15
2.2 Scalar Linear Recurrences	15
2.3 Impulse Response	16
2.4 Vector Linear Recurrences	16
2.5 Time Invariance	17
2.6 Stability In Discrete Time	17
2.7 Diagonalization And Modes	18
2.8 Complex Eigenvalues And Oscillation	18
2.9 Vanishing, Exploding, And Long Memory	19
2.10 Transfer-Function Intuition	19
2.11 Finite Versus Infinite Filters	20

2.12	Multiple Inputs And Outputs	20
2.13	What Is Linear, And What Is Learned?	20
2.14	A Useful Mental Picture	21
2.15	What To Remember	21
2.16	Exercises	21
2.17	Prepares You For	21
3	State Space Models	22
3.1	What Is A State Space Model?	22
3.2	Why Continuous Time?	22
3.3	Solving The Continuous Linear System	23
3.4	Discretization	23
3.5	Step Size	24
3.6	Continuous And Discrete Stability	24
3.7	Recurrent Mode	24
3.8	Convolution Mode	25
3.9	The Role Of The Direct Term	25
3.10	SISO And MIMO	25
3.11	State Size, Model Width, And Channels	26
3.12	A Deep SSM Layer Is Not Just The SSM	26
3.13	Why Structure Is Needed	26
3.14	SSMs Versus RNNs	27
3.15	SSMs Versus CNNs	27
3.16	SSMs Versus Transformers	28
3.17	What To Remember	28
3.18	Exercises	28
3.19	Prepares You For	28
4	HiPPO And Memory	29
4.1	Why HiPPO Appears In SSM Papers	29
4.2	The Online Memory Problem	29
4.3	Projection Onto A Basis	30
4.4	A Quick Reminder On Orthogonality	30
4.5	Why The Basis Changes Over Time	31
4.6	Legendre Memory Intuition	31
4.7	Why This Helps Long Memory	31
4.8	HiPPO As An SSM Initialization	32
4.9	The HiPPO Matrix Shape	32
4.10	Measures And Weighting The Past	32
4.11	What HiPPO Does Not Solve	33
4.12	Connection To Neural Time Series	33
4.13	A Minimal Worked Picture	33
4.14	What To Remember	34
4.15	Exercises	34
4.16	Prepares You For	34

5	S4	35
5.1	What Problem Did S4 Solve?	35
5.2	Why Ordinary SSMs Are Not Enough	35
5.3	The S4 Layer At A High Level	36
5.4	SISO SSMs In S4	36
5.5	HiPPO Initialization In S4	36
5.6	Normal Matrices	37
5.7	Normal Plus Low-Rank Structure	37
5.8	Complex Diagonal Dynamics	38
5.9	Generating The Convolution Kernel	38
5.10	The Cauchy Kernel Idea	38
5.11	Convolutional Training	39
5.12	Recurrent Inference	39
5.13	Discretization In S4	39
5.14	The S4 Block	40
5.15	What S4 Learns	40
5.16	Why S4 Was Important	40
5.17	Limitations And Complexity	40
5.18	A Reading Strategy For The S4 Paper	41
5.19	What To Remember	41
5.20	Exercises	41
5.21	Prepares You For	41
6	S5	42
6.1	Why S5 Exists	42
6.2	The S5 Recurrence	42
6.3	Diagonal Does Not Mean Trivial	43
6.4	Complex Diagonal Modes	43
6.5	MIMO State Space Layers	43
6.6	Discretization And Learned Timescales	44
6.7	Parallel Scan	44
6.8	Scan Versus Convolution	45
6.9	Initialization From S4 And HiPPO	45
6.10	The S5 Block	45
6.11	State Size And Model Width	45
6.12	S5 Compared With S4	46
6.13	Why S5 Is Interesting For Neural Data	46
6.14	Limitations	46
6.15	A Reading Strategy For The S5 Paper	47
6.16	What To Remember	47
6.17	Exercises	47
6.18	Prepares You For	47
7	Mamba	48
	Purpose	48
	Assumed Background	48
	Key Concepts To Define	48
	Primary Sources	48

Prepares You For	48
8 Comparison And Project Relevance	49
Purpose	49
Architectures To Compare	49
Comparison Axes	49
Project Connection	49
Exit Criteria For The Notes	50

Preface

These notes are written for a reader who already understands ordinary RNNs, transformers, back-propagation, regularization, and standard deep-learning training practice. The goal is to rebuild the missing bridge from convolutional sequence models to modern deep state space models: S4 ([Gu et al., 2022](#)), S5 ([Smith et al., 2023](#)), and Mamba ([Gu and Dao, 2023](#)).

The intended level is graduate-style: enough linear algebra, signal-processing language, and implementation detail to read the original papers directly, while keeping derivations in service of model understanding. The final chapter connects the architecture ideas back to self-supervised Utah array sequence modeling.

Chapter 1

Convolutions For Sequences

1.1 Why Start With Convolutions?

Modern state space models are often introduced as alternatives to RNNs and transformers, but the cleanest entry point is convolution. S4, S5, and related models can all be read as ways to construct long, structured sequence filters. Before worrying about HiPPO matrices, diagonal state transitions, or selective scans, we should be comfortable with this simpler idea:

a sequence model can be a learned filter applied across time.

You already know the transformer view: each token representation is updated by mixing information from other positions. You also know the RNN view: a hidden state carries a summary forward one step at a time. The convolutional view is different. It says that the output at time t can be computed from a fixed local pattern around t , using the same weights at every time. This weight sharing is what makes convolution useful, and it is also what gives us the first bridge to state space models.

The CNN vocabulary in this chapter is standard; see [Goodfellow et al. \(2016\)](#) and the CS231n notes ([Li et al., 2024](#)) for the broader image-modeling treatment. Here we immediately specialize it to one-dimensional temporal sequences, because that is the setting needed for deep SSMS and for neural time series.

1.2 Sequences, Channels, And Shapes

Let an input sequence be

$$\mathbf{u}_1, \mathbf{u}_2, \dots, \mathbf{u}_L, \quad \mathbf{u}_t \in \mathbb{R}^{C_{\text{in}}},$$

where L is the sequence length and C_{in} is the number of input channels. For neural data, a channel might be a spike-count feature, a threshold-crossing feature, or a spike-band-power feature. For language, it might be one coordinate of an embedding. For audio, it might be one feature in a spectrogram frame.

A one-dimensional convolutional layer maps this sequence to

$$\mathbf{y}_1, \mathbf{y}_2, \dots, \mathbf{y}_{L'}, \quad \mathbf{y}_t \in \mathbb{R}^{C_{\text{out}}},$$

where C_{out} is the number of output channels and L' depends on padding, stride, and kernel size. In deep-learning libraries, a batch of temporal data is often stored as one of the following:

$$(B, L, C) \quad \text{or} \quad (B, C, L),$$

where B is batch size. The math is the same either way, but being explicit about the time axis prevents many implementation mistakes.

1.3 Convolution Versus Cross-Correlation

Textbooks define discrete convolution between scalar sequences u and k as

$$(k * u)_t = \sum_i k_i u_{t-i}.$$

The index $t - i$ means the kernel is flipped relative to the input. Many deep-learning libraries actually implement cross-correlation:

$$(k \star u)_t = \sum_i k_i u_{t+i}.$$

The distinction matters in signal processing, but it is usually not important for learned CNNs because the kernel values are trainable. If the model can learn k_0, k_1, k_2 , it can also learn the reversed ordering. In these notes, we will use the causal sequence-modeling convention

$$\mathbf{y}_t = \sum_{i=0}^{K-1} \mathbf{K}_i \mathbf{u}_{t-i}. \quad (1.1)$$

This is a true past-looking filter: $\mathbf{y}_t$ depends on $\mathbf{u}_t, \mathbf{u}_{t-1}, \dots, \mathbf{u}_{t-K+1}$ and not on future inputs.

The kernel slice $\mathbf{K}_i$ is a matrix

$$\mathbf{K}_i \in \mathbb{R}^{C_{\text{out}} \times C_{\text{in}}},$$

so (1.1) means

$$\mathbf{y}_t[c_{\text{out}}] = \sum_{i=0}^{K-1} \sum_{c_{\text{in}}=1}^{C_{\text{in}}} \mathbf{K}_i[c_{\text{out}}, c_{\text{in}}] \mathbf{u}_{t-i}[c_{\text{in}}].$$

This is the multi-channel version of a finite impulse-response filter. The layer uses the same kernel matrices at every time t , which is the temporal weight-sharing assumption.

1.4 Causality

For offline image classification, a convolution can use pixels on both sides of a location. For a causal decoder, that would be future leakage. A causal model must produce the output at time t using only information available at or before t :

$$\mathbf{y}_t = f(\mathbf{u}_1, \dots, \mathbf{u}_t).$$

Equation (1.1) satisfies this by construction. A centered convolution such as

$$\mathbf{y}_t = \mathbf{K}_{-1} \mathbf{u}_{t-1} + \mathbf{K}_0 \mathbf{u}_t + \mathbf{K}_1 \mathbf{u}_{t+1}$$

does not. It may be perfectly appropriate for offline smoothing or bidirectional representation learning, but it is not causal.

This distinction will stay important throughout the notes. S5 and Mamba can both be run causally. Some training objectives or architectural wrappers can still accidentally leak future information if masking, patching, padding, or normalization is handled carelessly.

1.5 Padding And Boundary Effects

The formula

$$\mathbf{y}_t = \sum_{i=0}^{K-1} \mathbf{K}_i \mathbf{u}_{t-i}$$

refers to inputs such as $\mathbf{u}_0, \mathbf{u}_{-1}$ near the beginning of the sequence. A padding rule defines what happens there.

Valid convolution. Only compute outputs when the full window exists. For a causal kernel of length K , this starts at $t = K$. There is no artificial boundary value, but the output sequence is shorter.

Zero padding. Set out-of-range inputs to zero. This preserves length, but early outputs see many zeros. That is often fine, but the model can learn boundary artifacts if the sequence start has special meaning.

Replication or reflection padding. Use repeated or reflected boundary values. These are common in image processing, less central for causal sequence modeling, and often inappropriate for online decoding because reflection can use future values.

For causal sequence models, the usual length-preserving choice is left zero padding:

$$\mathbf{u}_t = 0 \quad \text{for } t \leq 0.$$

Then $\mathbf{y}_1$ is computed from mostly padding and $\mathbf{u}_1$, while later positions see a full history window. In long sequences the boundary region is usually negligible, but in short segments or patch-based training it can be a real source of distribution shift.

1.6 Stride And Downsampling

Stride means the layer does not produce an output at every input time. With stride s , one common causal convention is

$$\mathbf{y}_j = \sum_{i=0}^{K-1} \mathbf{K}_i \mathbf{u}_{js-i}.$$

If $s = 2$, the layer emits one output for every two input steps. Stride is therefore both a computational choice and a modeling choice: it reduces temporal resolution.

For image CNNs, stride is often a spatial downsampling tool. For temporal neural data, it also changes the time scale of the representation. If bins are 20 ms wide, stride 5 means each output step advances by 100 ms. That may be useful for some high-level features, but it can be too coarse for precise speech timing.

Stride should not be confused with kernel size. A kernel of size $K = 5$ with stride 1 looks at five time bins and advances by one bin. A kernel of size $K = 5$ with stride 5 looks at five time bins and advances by five bins. Both have the same local window, but the second throws away intermediate output positions.

1.7 Receptive Field

The receptive field of an output position is the set of input positions that can affect it. For a single causal convolution with kernel size K and stride 1, the receptive field of $\mathbf{y}_t$ is

$$\{\mathbf{u}_t, \mathbf{u}_{t-1}, \dots, \mathbf{u}_{t-K+1}\},$$

so its size is K .

Stacking convolutional layers increases the receptive field. Suppose we stack N causal convolutions, each with kernel size K and stride 1. Then the final output at time t depends on

$$1 + N(K - 1)$$

input positions. For example, five layers of kernel size 3 see

$$1 + 5(3 - 1) = 11$$

positions. This linear growth is a limitation if we need long context.

This is one reason long-sequence models become interesting. A transformer can access all previous tokens in one layer under causal attention, but with quadratic attention cost in sequence length. A plain CNN is cheap and parallel, but a shallow one sees only local context. Deep SSMs try to get long effective memory with linear or near-linear cost.

1.8 Dilated Convolution

Dilated convolution expands receptive field without increasing kernel size. A causal dilated convolution with dilation d is

$$\mathbf{y}_t = \sum_{i=0}^{K-1} \mathbf{K}_i \mathbf{u}_{t-di}.$$

If $d = 1$, this is the ordinary causal convolution. If $d = 4$ and $K = 3$, the output uses $\mathbf{u}_t$, $\mathbf{u}_{t-4}$, and $\mathbf{u}_{t-8}$.

Stacking layers with increasing dilation can grow the receptive field quickly. A common pattern is

$$d = 1, 2, 4, 8, \dots$$

This gives a large context using few layers, and is one reason dilated temporal CNNs have been successful in audio and sequence modeling.

The tradeoff is that dilation samples the past on a sparse grid inside each layer. Later layers can combine information across these grids, but the architecture still imposes a particular multiscale pattern. State space models will take a different route: they use a recurrence to generate a long dense filter, often with smoothly decaying and oscillatory modes.

1.9 Channel Mixing

The scalar convolution formula hides an important issue: how do features interact across channels? In a standard multi-channel convolution, every output channel can use every input channel at every kernel offset:

$$\mathbf{K}_i \in \mathbb{R}^{C_{\text{out}} \times C_{\text{in}}}.$$

The number of parameters is

$$KC_{\text{in}}C_{\text{out}}.$$

This is expressive, but expensive when both channel counts are large.

1.9.1 Pointwise Convolution

A pointwise convolution is a kernel of size 1:

$$\mathbf{y}_t = \mathbf{W} \mathbf{u}_t, \quad \mathbf{W} \in \mathbb{R}^{C_{\text{out}} \times C_{\text{in}}}.$$

It mixes channels but does not mix time. In sequence models, pointwise layers are often used before or after a temporal operation. For example, a block might project from model width H to an expanded hidden width, run a temporal operation, then project back.

1.9.2 Depthwise Convolution

A depthwise convolution applies a separate temporal filter to each channel:

$$y_t[c] = \sum_{i=0}^{K-1} k_i[c] u_{t-i}[c].$$

There is temporal mixing within each channel, but no channel mixing. The parameter count is only $K C_{\text{in}}$ if $C_{\text{out}} = C_{\text{in}}$.

Depthwise convolution is often paired with pointwise convolution:

$$\text{channel mixing} \rightarrow \text{depthwise temporal mixing} \rightarrow \text{channel mixing}.$$

This factorization is less expressive than a full convolution but much cheaper. Mamba uses a small local convolution inside its block, and this kind of separation between local temporal processing and channel projection is a recurring design pattern.

1.9.3 Grouped Convolution

Grouped convolution lies between full and depthwise convolution. Channels are partitioned into groups, and convolution is full within each group but not across groups. This reduces parameter count and computation while allowing more channel interaction than a purely depthwise operation.

1.10 Convolution As A Matrix

For a finite sequence, convolution can be written as multiplication by a structured matrix. In the scalar causal case with kernel (k_0, k_1, k_2) and left zero padding,

$$\begin{bmatrix} y_1 \\ y_2 \\ y_3 \\ y_4 \end{bmatrix} = \begin{bmatrix} k_0 & 0 & 0 & 0 \\ k_1 & k_0 & 0 & 0 \\ k_2 & k_1 & k_0 & 0 \\ 0 & k_2 & k_1 & k_0 \end{bmatrix} \begin{bmatrix} u_1 \\ u_2 \\ u_3 \\ u_4 \end{bmatrix}.$$

This is a lower-triangular Toeplitz matrix: each diagonal is constant. The lower-triangular structure is causality. The constant diagonals are time invariance.

This matrix view is useful for three reasons.

1. It makes convolution look like a linear layer with heavy weight sharing.
2. It shows why convolution is parallel over time: all outputs are part of one structured matrix-vector product.

3. It prepares us for SSMs, where the same kind of Toeplitz convolution matrix can be generated from recurrence parameters rather than learned directly.

Of course, deep-learning libraries do not literally build this large matrix for ordinary small convolutions. They use specialized kernels. But mathematically, the structured matrix picture is accurate.

1.11 Finite Impulse Response Filters

A causal convolution with kernel length K is a finite impulse-response filter, usually abbreviated FIR. To see this, imagine a scalar input sequence that is zero everywhere except at time 1:

$$u_1 = 1, \quad u_t = 0 \text{ for } t \neq 1.$$

For a causal convolution,

$$y_1 = k_0, \quad y_2 = k_1, \quad \dots, \quad y_K = k_{K-1},$$

and then the response becomes zero. The kernel is exactly the impulse response.

This language matters because linear state space models produce impulse responses too. The difference is that an SSM impulse response can be very long, even infinite in principle, while being parameterized compactly by state matrices.

1.12 Convolution And Parallelism

A vanilla RNN computes

$$\mathbf{h}_t = f(\mathbf{h}_{t-1}, \mathbf{u}_t).$$

This creates a sequential dependency: to compute $\mathbf{h}_t$, we first need $\mathbf{h}_{t-1}$. That does not mean RNNs cannot be optimized, but it does limit straightforward parallelism over time.

A convolution computes

$$\mathbf{y}_t = \sum_{i=0}^{K-1} \mathbf{K}_i \mathbf{u}_{t-i}.$$

Once the input sequence and kernel are known, every $\mathbf{y}_t$ can be computed independently of every other $\mathbf{y}_{t'}$. The outputs share weights, but they do not depend on one another. This is why CNNs train efficiently on long sequences.

The catch is memory length. A small convolution has a short receptive field. A large convolution can have a long receptive field, but a directly learned dense kernel of length L has many parameters and can be expensive. The S4/S5 idea is to get the benefits of long convolution while using a structured parameterization inherited from dynamical systems.

1.13 FFT Convolution

There is another reason convolution is computationally special: long convolutions can be accelerated using the Fourier transform. The convolution theorem says, informally,

$$\mathcal{F}(k * u) = \mathcal{F}(k) \odot \mathcal{F}(u),$$

where $\odot$ is elementwise multiplication. So a long convolution can be computed by:

$$\text{FFT} \rightarrow \text{elementwise multiply} \rightarrow \text{inverse FFT}.$$

For very long kernels, this can be much cheaper than directly summing over every offset.

This is one reason the convolutional view of SSMs is valuable. If an SSM can produce a long kernel $\mathbf{K}_0, \dots, \mathbf{K}_{L-1}$, then training can sometimes use fast convolution methods instead of stepping the recurrence one time step at a time. S4 famously leans on this distinction: recurrent for online generation, convolutional for parallel training.

There are details we are postponing. FFT convolution requires care with padding because the discrete Fourier transform naturally computes circular convolution. To get ordinary linear convolution, one pads the sequences before transforming. The key idea for now is only that convolution has global algorithms that exploit structure.

1.14 Convolutional Sequence Blocks

A single linear convolution is usually not enough. Neural sequence models use convolution inside nonlinear blocks. A simple temporal CNN block might look like

$$\mathbf{u}_{1:L} \rightarrow \text{LayerNorm} \rightarrow \text{causal convolution} \rightarrow \text{nonlinearity} \rightarrow \text{pointwise projection} \rightarrow \text{residual addition}.$$

The exact normalization and activation choices vary. The important point is that the temporal operation is embedded in a deep residual architecture, just as attention is embedded in transformer blocks and SSM layers are embedded in S4/S5/Mamba blocks.

A residual block typically has the schematic form

$$\mathbf{z}_{1:L}^{(n+1)} = \mathbf{z}_{1:L}^{(n)} + F_{\theta}^{(n)}(\mathbf{z}_{1:L}^{(n)}),$$

where $F_{\theta}^{(n)}$ contains the temporal mixing operation plus channel mixing and nonlinearities. This residual structure is not unique to CNNs, but it is essential for deep sequence models because it stabilizes optimization and lets layers refine rather than completely replace representations.

1.15 CNNs, RNNs, And Transformers

It is useful to compare the three familiar sequence-modeling patterns before introducing SSMs.

Model family	Temporal mixing	Parallel over time?	Context pattern
CNN	fixed kernel	yes	local or dilated
RNN	hidden recurrence	no, in ordinary form	compressed past state
Transformer	attention weights	yes, during training	direct pairwise access

Table 1.1: A simplified comparison of common sequence-modeling mechanisms.

CNNs are efficient because the same local filter is applied everywhere. RNNs are naturally online because they update a state one step at a time. Transformers are flexible because attention weights depend on the content of the sequence, but causal self-attention over length L has an $L \times L$ interaction pattern.

Deep SSMs occupy a middle ground. Like RNNs, they maintain a state and can be run online. Like CNNs, linear time-invariant SSMs can be expressed as convolutions and trained in parallel. Like neither plain CNNs nor plain RNNs, modern SSMs use carefully structured state matrices to produce long, stable filters.

1.16 The Bridge To State Space Models

The most important equation in this chapter is still the causal convolution:

$$\mathbf{y}_t = \sum_{i=0}^{K-1} \mathbf{K}_i \mathbf{u}_{t-i}.$$

For an ordinary CNN, the matrices $\mathbf{K}_i$ are learned directly as free parameters. If $K = 7$, the model learns seven kernel slices. If $K = 1024$, direct learning becomes much heavier.

A linear state space model instead defines a recurrence

$$\mathbf{x}_t = \mathbf{A}\mathbf{x}_{t-1} + \mathbf{B}\mathbf{u}_t, \quad \mathbf{y}_t = \mathbf{C}\mathbf{x}_t + \mathbf{D}\mathbf{u}_t.$$

If we unroll this recurrence, we get a convolution-like expression:

$$\mathbf{y}_t = \mathbf{D}\mathbf{u}_t + \sum_{i=0}^{t-1} \mathbf{C}\mathbf{A}^i \mathbf{B}\mathbf{u}_{t-i}.$$

The kernel is no longer directly learned as arbitrary matrices. It is generated by

$$\mathbf{K}_i = \mathbf{C}\mathbf{A}^i \mathbf{B}.$$

This is the central bridge. A CNN says, “learn the filter.” An SSM says, “learn a dynamical system whose impulse response is the filter.” S4, S5, and Mamba differ in how they parameterize this dynamical system, how they compute it efficiently, and whether the parameters are fixed across time or depend on the input.

1.17 What To Remember

- A temporal convolution is a shared filter applied across sequence positions.
- Causal convolution only uses present and past inputs.
- Kernel size, stride, dilation, and depth determine the receptive field.
- Full, depthwise, grouped, and pointwise convolutions make different tradeoffs between temporal mixing and channel mixing.
- Convolution can be represented as multiplication by a structured Toeplitz matrix.
- A finite convolution kernel is an impulse response.
- SSMs generate convolution kernels from recurrence parameters, which is why this chapter is the right starting point for S4 and S5.

1.18 Exercises

1. For a stack of four causal convolutions with kernel size 5, stride 1, and no dilation, compute the receptive field size.
2. Write the Toeplitz matrix for a scalar causal convolution with kernel (k_0, k_1, k_2, k_3) applied to a length-5 input sequence with left zero padding.

3. Suppose neural data are binned at 20 ms. What time span is covered by a causal convolution with kernel size 9? What changes if the dilation is 2?
4. Explain in words why a causal convolution is parallel during training but still respects online decoding constraints.
5. Compare a learned CNN kernel $\mathbf{K}_i$ with an SSM-generated kernel $\mathbf{C}\mathbf{A}^i\mathbf{B}$. What is gained and what is constrained by the SSM parameterization?

Chapter 2

Linear Systems And Filters

2.1 Why Linear Systems Matter

Chapter 1 ended with the key bridge: a convolution kernel can be learned directly, or it can be generated by a recurrence. This chapter explains that bridge more carefully.

The models in this chapter are linear. That may sound too weak for deep learning, but it is exactly the right simplification. Deep sequence architectures often alternate between linear temporal mixing, nonlinear pointwise transformations, normalization, and residual connections. Attention is mostly a learned mixing operation inside a nonlinear block. Convolution is a linear temporal filter inside a nonlinear block. S4 and S5 are also linear temporal filters inside nonlinear blocks.

So the question is not whether a whole neural network should be linear. It should not. The question is how much temporal structure can be captured by a powerful linear module when that module is embedded in a deep nonlinear architecture.

2.2 Scalar Linear Recurrences

Start with the simplest possible recurrence:

$$x_t = ax_{t-1} + bu_t, \quad y_t = cx_t + du_t, \quad (2.1)$$

where u_t is the input, x_t is the state, and y_t is the output. The parameters are scalars a, b, c, d . Assume $x_0 = 0$ for now.

Unrolling gives

$$\begin{aligned} x_1 &= bu_1, \\ x_2 &= abu_1 + bu_2, \\ x_3 &= a^2bu_1 + abu_2 + bu_3. \end{aligned}$$

In general,

$$x_t = \sum_{i=0}^{t-1} a^i bu_{t-i}.$$

Therefore

$$y_t = du_t + c \sum_{i=0}^{t-1} a^i bu_{t-i}.$$

This is already a convolution. Its kernel is

$$k_0 = cb + d, \quad k_i = ca^i b \quad \text{for } i \geq 1,$$

up to the convention for whether the direct term du_t is folded into k_0 .

This scalar recurrence produces a geometric impulse response:

$$b, ab, a^2b, a^3b, \dots$$

If $|a| < 1$, the memory decays. If $|a| > 1$, the response grows. If a is close to 1, the memory decays slowly. This is the first appearance of a theme that will keep returning: long memory requires dynamics that do not vanish too quickly, but stable training requires dynamics that do not explode.

2.3 Impulse Response

The impulse response is the output produced by a unit input at one time and zero input elsewhere. For the scalar recurrence above, let

$$u_1 = 1, \quad u_t = 0 \text{ for } t > 1, \quad x_0 = 0.$$

Then

$$x_1 = b, \quad x_2 = ab, \quad x_3 = a^2b, \quad \dots$$

and, ignoring the direct term for $t > 1$,

$$y_t = ca^{t-1}b.$$

For a finite CNN kernel, the impulse response becomes exactly zero after K steps. For the recurrence, the impulse response can continue forever:

$$cb, cab, ca^2b, ca^3b, \dots$$

This is an infinite impulse-response filter, usually abbreviated IIR. A finite convolutional kernel is an FIR filter. Linear recurrences naturally give IIR filters.

In practice, a stable IIR response may become numerically negligible after some time, but it is not forced to have a hard cutoff. This is one reason recurrences are attractive for long-context modeling.

2.4 Vector Linear Recurrences

Now move from a scalar state to a vector state:

$$\mathbf{x}_t = \mathbf{A}\mathbf{x}_{t-1} + \mathbf{B}\mathbf{u}_t, \quad \mathbf{y}_t = \mathbf{C}\mathbf{x}_t + \mathbf{D}\mathbf{u}_t. \quad (2.2)$$

Here

$$\mathbf{x}_t \in \mathbb{R}^N, \quad \mathbf{u}_t \in \mathbb{R}^{C_{\text{in}}}, \quad \mathbf{y}_t \in \mathbb{R}^{C_{\text{out}}},$$

and the parameter shapes are

$$\mathbf{A} \in \mathbb{R}^{N \times N}, \quad \mathbf{B} \in \mathbb{R}^{N \times C_{\text{in}}}, \quad \mathbf{C} \in \mathbb{R}^{C_{\text{out}} \times N}, \quad \mathbf{D} \in \mathbb{R}^{C_{\text{out}} \times C_{\text{in}}}.$$

The state dimension N is not necessarily the same as the input channel count or the model width. It is the size of the dynamical memory. In SSM papers this is often called the state size.

Assuming $\mathbf{x}_0 = 0$, unrolling gives

$$\mathbf{x}_t = \mathbf{B}\mathbf{u}_t + \mathbf{A}\mathbf{B}\mathbf{u}_{t-1} + \mathbf{A}^2\mathbf{B}\mathbf{u}_{t-2} + \cdots + \mathbf{A}^{t-1}\mathbf{B}\mathbf{u}_1.$$

Substituting into the output equation,

$$\mathbf{y}_t = \mathbf{D}\mathbf{u}_t + \sum_{i=0}^{t-1} \mathbf{C}\mathbf{A}^i\mathbf{B}\mathbf{u}_{t-i}. \quad (2.3)$$

So the recurrence defines a convolution kernel

$$\mathbf{K}_i = \mathbf{C}\mathbf{A}^i\mathbf{B}. \quad (2.4)$$

This is the central formula for linear state space sequence models.

2.5 Time Invariance

The recurrence in (2.2) uses the same matrices $\mathbf{A}, \mathbf{B}, \mathbf{C}, \mathbf{D}$ at every time step. Such a system is time invariant. If the same input pattern occurs later in the sequence, the system responds the same way, shifted in time, assuming the same initial state context.

Time invariance is what makes the convolution representation possible. The kernel $\mathbf{K}_i$ depends only on the lag i , not on the absolute time t . If the matrices changed with time,

$$\mathbf{x}_t = \mathbf{A}_t\mathbf{x}_{t-1} + \mathbf{B}_t\mathbf{u}_t,$$

then the effect of $\mathbf{u}_{t-i}$ on $\mathbf{y}_t$ would depend on all the intermediate matrices. There would no longer be a single fixed convolution kernel.

This is one major distinction between S4/S5 and Mamba. S4 and S5 are built around linear time-invariant SSMs inside a neural block. Mamba introduces input-dependent parameters, which breaks the fixed convolution view but adds content-dependent selectivity.

2.6 Stability In Discrete Time

To understand stability, ignore the input and direct output:

$$\mathbf{x}_t = \mathbf{A}\mathbf{x}_{t-1}.$$

Then

$$\mathbf{x}_t = \mathbf{A}^t\mathbf{x}_0.$$

The long-term behavior is controlled by powers of $\mathbf{A}$. If these powers grow, the system is unstable. If they decay, the system forgets its initial state.

For a scalar recurrence $x_t = ax_{t-1}$:

$$x_t = a^t x_0.$$

The behavior is simple:

$$\begin{aligned} |a| < 1 & \text{ decay,} \\ |a| = 1 & \text{ persistent magnitude, possibly oscillatory,} \\ |a| > 1 & \text{ growth.} \end{aligned}$$

For a matrix $\mathbf{A}$, the analogous quantities are eigenvalues. If λ is an eigenvalue with eigenvector $\mathbf{v}$, then

$$\mathbf{A}\mathbf{v} = \lambda\mathbf{v}.$$

Along that eigenvector direction,

$$\mathbf{A}^t\mathbf{v} = \lambda^t\mathbf{v}.$$

Thus, the magnitude $|\lambda|$ controls growth or decay of that mode. A discrete-time linear system is stable, roughly, when all eigenvalues of $\mathbf{A}$ lie inside the unit circle:

$$|\lambda_j(\mathbf{A})| < 1 \quad \text{for all } j.$$

This is the discrete-time version. Continuous-time systems use a different stability condition: eigenvalues should have negative real part. Chapter 3 will connect these two views through discretization.

2.7 Diagonalization And Modes

Suppose $\mathbf{A}$ can be diagonalized:

$$\mathbf{A} = \mathbf{V}\mathbf{\Lambda}\mathbf{V}^{-1},$$

where

$$\mathbf{\Lambda} = \text{diag}(\lambda_1, \dots, \lambda_N).$$

Then

$$\mathbf{A}^i = \mathbf{V}\mathbf{\Lambda}^i\mathbf{V}^{-1},$$

and

$$\mathbf{\Lambda}^i = \text{diag}(\lambda_1^i, \dots, \lambda_N^i).$$

This means the dynamics decompose into independent modes in the eigenbasis. Each mode has its own decay rate and possible oscillation.

Using (2.4),

$$\mathbf{K}_i = \mathbf{C}\mathbf{V}\mathbf{\Lambda}^i\mathbf{V}^{-1}\mathbf{B}.$$

So the kernel is a mixture of terms like λ_j^i . In the scalar case, there was only one geometric sequence. In the vector case, the impulse response is a learned mixture of many geometric sequences.

This is a useful mental model for SSMs:

$$\text{SSM kernel} \approx \text{mixture of decays and oscillations.}$$

Some modes can remember slowly. Some can respond quickly. Some can oscillate. The model learns how input channels excite these modes through $\mathbf{B}$ and how modes are read out through $\mathbf{C}$.

2.8 Complex Eigenvalues And Oscillation

Real matrices can have complex eigenvalues. For real-valued systems, complex eigenvalues come in conjugate pairs:

$$\lambda = re^{i\omega}, \quad \bar{\lambda} = re^{-i\omega}.$$

Powers of λ behave as

$$\lambda^t = r^t e^{i\omega t}.$$

The magnitude r^t controls decay or growth. The angle ωt controls oscillation.

This is why complex-valued diagonal SSM parameterizations are natural. A mode with r close to 1 and nonzero ω gives a slowly decaying oscillatory filter. Such filters are useful for temporal data because real signals often have rhythms, periodic components, and repeated temporal patterns.

The model output can still be real-valued. Complex conjugate modes combine to produce real sines and cosines. In implementation, one may store complex parameters directly or use equivalent real block matrices. The papers often use complex notation because it makes the modal structure much cleaner.

2.9 Vanishing, Exploding, And Long Memory

The same eigenvalue logic explains the old RNN problem of vanishing and exploding gradients. In a recurrent computation, forward signals and backward gradients both involve repeated multiplication by transition-like matrices. Repeated powers can shrink or grow rapidly.

If all eigenvalues have magnitudes much smaller than 1, the system forgets quickly. That is stable but short-memory. If eigenvalues have magnitudes larger than 1, the system can explode. If eigenvalues are very close to the unit circle, the system can keep long memory, but optimization becomes delicate.

The goal is not simply to put every eigenvalue at 1. That would preserve information but could make the system brittle and poorly conditioned. The goal is to have a useful spectrum of timescales:

some fast modes + some medium modes + some slow modes.

HiPPO and S4-style initializations can be understood as principled attempts to initialize such a spectrum for long-range sequence modeling.

2.10 Transfer-Function Intuition

Signal processing often describes linear systems by transfer functions. We do not need the full machinery yet, but the intuition is helpful.

For a scalar convolution kernel $k_0, k_1, k_2, \dots$, define the formal series

$$H(z) = \sum_{i=0}^{\infty} k_i z^{-i}.$$

This says how the system weights different temporal frequencies. For a state space model, the transfer function has the form

$$H(z) = \mathbf{C}(z\mathbf{I} - \mathbf{A})^{-1}\mathbf{B} + \mathbf{D},$$

up to convention choices about indexing. This compact expression contains the same information as the impulse response $\mathbf{K}_i = \mathbf{C}\mathbf{A}^i\mathbf{B}$.

You do not need to manipulate transfer functions fluently to understand S4 or S5. The important idea is that linear systems can be described in multiple equivalent ways:

recurrence $\leftrightarrow$ impulse response $\leftrightarrow$ convolution $\leftrightarrow$ frequency response.

Deep SSM papers move between these views because each one makes a different computational or modeling property visible.

2.11 Finite Versus Infinite Filters

CNNs usually learn finite kernels:

$$\mathbf{K}_0, \dots, \mathbf{K}_{K-1}.$$

The parameter count grows linearly with kernel length K . If one wants a kernel that can cover thousands of positions, direct learning becomes unattractive.

Linear systems generate kernels through matrix powers:

$$\mathbf{K}_i = \mathbf{C}\mathbf{A}^i\mathbf{B}.$$

The parameter count depends on the state size and channel dimensions, not directly on the sequence length. A compact state can generate a very long impulse response. This is the basic compression advantage of a state space model.

There is also a constraint. Not every arbitrary long kernel can be represented well by a small state. The SSM kernel must have structure: it is built from modes of $\mathbf{A}$. That structure is the inductive bias. It is helpful when the sequence really does contain smooth, persistent, or oscillatory temporal dynamics. It may be limiting when the task requires arbitrary content-dependent interactions.

2.12 Multiple Inputs And Outputs

In scalar examples, there is one input and one output. Real neural networks use many channels. Equation (2.2) handles this with $\mathbf{B}, \mathbf{C}, \mathbf{D}$.

The matrix $\mathbf{B}$ decides how input channels drive the state:

$$\mathbf{B}\mathbf{u}_t \in \mathbb{R}^N.$$

The matrix $\mathbf{C}$ decides how the state is read into output channels:

$$\mathbf{C}\mathbf{x}_t \in \mathbb{R}^{C_{\text{out}}}.$$

The matrix $\mathbf{D}$ gives an immediate input-to-output path:

$$\mathbf{D}\mathbf{u}_t.$$

This distinction becomes important in S5. S4 often uses many single-input single-output SSMs plus channel mixing around them. S5 emphasizes multi-input multi-output SSMs, where one state space layer can jointly map multiple input channels to multiple output channels.

2.13 What Is Linear, And What Is Learned?

The recurrence in (2.2) is linear in the input sequence once the parameters are fixed. But the parameters $\mathbf{A}, \mathbf{B}, \mathbf{C}, \mathbf{D}$ are learned by gradient descent as part of a larger neural network.

In a deep block, the SSM may be surrounded by:

normalization, linear projections, gates, nonlinearities, residual connections.

Thus the full model is nonlinear even if the temporal core is linear. This is exactly analogous to a CNN: a convolution layer is linear, but a CNN block with activations and residual paths is not.

2.14 A Useful Mental Picture

Think of the state $\mathbf{x}_t$ as a bank of leaky memory cells. Each mode remembers the past at a different rate, and possibly with a different oscillation. The input matrix $\mathbf{B}$ decides which memory cells are written to. The transition matrix $\mathbf{A}$ decides how those memories decay, rotate, and mix. The output matrix $\mathbf{C}$ decides which memories matter for the current output.

This picture is not mathematically complete, but it is useful. S4 and S5 are mostly about choosing a parameterization of this memory bank that is stable, expressive, and efficient for long sequences.

2.15 What To Remember

- A linear recurrence can be unrolled into a convolution.
- The impulse-response kernel of a discrete linear system is $\mathbf{K}_i = \mathbf{C}\mathbf{A}^i\mathbf{B}$.
- Eigenvalues of $\mathbf{A}$ control decay, growth, and oscillation.
- Long memory requires modes that decay slowly but remain stable.
- Diagonalization turns matrix powers into independent modal powers λ_j^i .
- Complex eigenvalues naturally represent decaying oscillations.
- SSMS trade arbitrary learned kernels for compact, structured, dynamically generated kernels.

2.16 Exercises

1. For the scalar recurrence $x_t = 0.9x_{t-1} + u_t$, $y_t = x_t$, write the first five impulse-response coefficients.
2. Repeat the previous exercise with $a = -0.9$. How does the sign change affect the impulse response?
3. Suppose $\mathbf{A} = \text{diag}(0.5, 0.9, 0.99)$. Which mode remembers longest? Which forgets fastest?
4. Show by unrolling that the vector recurrence $\mathbf{x}_t = \mathbf{A}\mathbf{x}_{t-1} + \mathbf{B}\mathbf{u}_t$ has terms $\mathbf{A}^i\mathbf{B}\mathbf{u}_{t-i}$.
5. Explain why making $\mathbf{A}$ input-dependent would break the existence of a single fixed convolution kernel.

2.17 Prepares You For

The next chapter formalizes state space models and explains how continuous-time dynamics are discretized for neural network sequence layers. The new ingredient will be the continuous system

$$\frac{d}{dt}\mathbf{x}(t) = \mathbf{A}\mathbf{x}(t) + \mathbf{B}\mathbf{u}(t),$$

and the question of how to convert it into a discrete recurrence suitable for sampled sequence data.

Chapter 3

State Space Models

3.1 What Is A State Space Model?

A state space model describes a system by separating its internal state from its observed input and output. The state is not just another feature vector. It is the model’s memory of the past, updated through a dynamical rule.

In continuous time, the standard linear state space model is

$$\frac{d}{dt}\mathbf{x}(t) = \mathbf{A}\mathbf{x}(t) + \mathbf{B}\mathbf{u}(t), \quad \mathbf{y}(t) = \mathbf{C}\mathbf{x}(t) + \mathbf{D}\mathbf{u}(t). \quad (3.1)$$

Here $\mathbf{u}(t)$ is the input signal, $\mathbf{x}(t)$ is the latent state, and $\mathbf{y}(t)$ is the output signal. The matrices have the same conceptual roles as in the discrete recurrence from Chapter 2:

$\mathbf{A}$: state dynamics, $\mathbf{B}$: input write-in, $\mathbf{C}$: state read-out, $\mathbf{D}$: direct input-output path.

In control theory, this notation is used to model physical systems. In deep learning, we use the same notation to build sequence layers. The state does not have to correspond to literal physical variables. It is a learned memory representation.

3.2 Why Continuous Time?

Neural networks usually operate on discrete sequences:

$$\mathbf{u}_1, \dots, \mathbf{u}_L.$$

So it is fair to ask why SSM papers start from continuous-time equations. There are three main reasons.

Sampling rate should be a parameter. Many signals come from continuous processes sampled at some interval. Neural recordings, audio, sensor streams, and trajectories all have a physical sampling interval. A continuous-time model can be discretized at different step sizes Δ .

Stability is cleaner in continuous time. For the continuous system

$$\frac{d}{dt}\mathbf{x}(t) = \mathbf{A}\mathbf{x}(t),$$

stability is controlled by the real parts of eigenvalues of $\mathbf{A}$. Negative real part means decay. This is often easier to parameterize before converting to discrete time.

S4 and HiPPO are naturally continuous. The HiPPO theory behind S4 begins as an online continuous-time memory problem. The continuous formulation is not just decoration; it is part of where the S4 initialization comes from.

After discretization, however, the model used in a neural network is an ordinary discrete sequence layer.

3.3 Solving The Continuous Linear System

To discretize (3.1), first understand the solution over one time interval. Assume the input $\mathbf{u}(t)$ is held constant over the interval $[t, t + \Delta)$. This assumption is called zero-order hold. It matches the common situation where a sampled input value represents the signal value for one bin.

The solution of

$$\frac{d}{dt}\mathbf{x}(t) = \mathbf{A}\mathbf{x}(t)$$

over a time step Δ is

$$\mathbf{x}(t + \Delta) = e^{\Delta\mathbf{A}}\mathbf{x}(t),$$

where $e^{\Delta\mathbf{A}}$ is the matrix exponential. This is the matrix analogue of the scalar solution

$$x(t + \Delta) = e^{\Delta a}x(t).$$

With a constant input over the interval, the solution becomes

$$\mathbf{x}(t + \Delta) = e^{\Delta\mathbf{A}}\mathbf{x}(t) + \left(\int_0^\Delta e^{\tau\mathbf{A}} d\tau\right)\mathbf{B}\mathbf{u}(t).$$

This formula gives the exact discrete update under zero-order hold.

3.4 Discretization

The discretized SSM has the form

$$\mathbf{x}_t = \bar{\mathbf{A}}\mathbf{x}_{t-1} + \bar{\mathbf{B}}\mathbf{u}_t, \quad \mathbf{y}_t = \mathbf{C}\mathbf{x}_t + \mathbf{D}\mathbf{u}_t. \quad (3.2)$$

The barred matrices are discrete-time versions of the continuous parameters. Under zero-order hold,

$$\bar{\mathbf{A}} = e^{\Delta\mathbf{A}}, \quad \bar{\mathbf{B}} = \left(\int_0^\Delta e^{\tau\mathbf{A}} d\tau\right)\mathbf{B}. \quad (3.3)$$

If $\mathbf{A}$ is invertible, the second expression can be written as

$$\bar{\mathbf{B}} = \mathbf{A}^{-1}(e^{\Delta\mathbf{A}} - \mathbf{I})\mathbf{B}.$$

The integral form is more general and conceptually safer.

The output matrices $\mathbf{C}$ and $\mathbf{D}$ are often left unchanged by discretization in the standard setup. The dynamics and input write-in need conversion because they describe how the state evolves over a time interval.

3.5 Step Size

The step size Δ determines how much continuous time passes between two sequence positions. If neural data are binned every 20 ms, then a physically literal choice is

$$\Delta = 0.02 \text{ seconds.}$$

In neural network SSM layers, Δ is often learned or parameterized per channel or per state dimension. Then it becomes more than a sampling constant: it controls the timescale of the learned dynamics.

For a scalar continuous system

$$\frac{d}{dt}x(t) = ax(t),$$

discretization gives

$$\bar{a} = e^{\Delta a}.$$

If $a < 0$, then $0 < \bar{a} < 1$. Larger Δ makes $\bar{a}$ smaller, meaning faster decay per discrete step. Smaller Δ keeps $\bar{a}$ closer to 1, meaning slower decay per step.

This is why learned step sizes are powerful. They let different channels or modes operate at different effective temporal scales.

3.6 Continuous And Discrete Stability

For the continuous autonomous system

$$\frac{d}{dt}\mathbf{x}(t) = \mathbf{A}\mathbf{x}(t),$$

a mode with eigenvalue λ evolves like

$$e^{\lambda t}.$$

If $\text{Re}(\lambda) < 0$, it decays. If $\text{Re}(\lambda) > 0$, it grows. If $\text{Re}(\lambda) = 0$, it persists in magnitude and may oscillate.

After discretization,

$$\bar{\lambda} = e^{\Delta \lambda}.$$

The magnitude is

$$|\bar{\lambda}| = |e^{\Delta \lambda}| = e^{\Delta \text{Re}(\lambda)}.$$

So

$$\text{Re}(\lambda) < 0 \iff |\bar{\lambda}| < 1.$$

This exactly connects the continuous-time stability condition to the discrete-time unit circle condition from Chapter 2.

3.7 Recurrent Mode

Once discretized, the SSM can be run as a recurrence:

$$\mathbf{x}_t = \bar{\mathbf{A}}\mathbf{x}_{t-1} + \bar{\mathbf{B}}\mathbf{u}_t, \quad \mathbf{y}_t = \mathbf{C}\mathbf{x}_t + \mathbf{D}\mathbf{u}_t.$$

This is the recurrent mode. It is natural for online inference because we only need to store the current state $\mathbf{x}_{t-1}$ and the new input $\mathbf{u}_t$.

For a causal neural decoder, recurrent mode is conceptually attractive:

new neural bin $\rightarrow$ update state $\rightarrow$ emit current representation.

There is no need to revisit the whole history.

The downside is training parallelism. A naive recurrence computes step t after step $t - 1$, just like a vanilla RNN. SSMs become especially useful when we exploit additional structure to train them in parallel.

3.8 Convolution Mode

If $\bar{\mathbf{A}}, \bar{\mathbf{B}}, \mathbf{C}, \mathbf{D}$ are fixed across time, the discretized SSM is linear and time invariant. As in Chapter 2, unrolling gives

$$\mathbf{y}_t = \mathbf{D}\mathbf{u}_t + \sum_{i=0}^{t-1} \mathbf{C}\bar{\mathbf{A}}^i\bar{\mathbf{B}}\mathbf{u}_{t-i}.$$

Define the SSM convolution kernel

$$\mathbf{K}_i = \mathbf{C}\bar{\mathbf{A}}^i\bar{\mathbf{B}}. \quad (3.4)$$

Then the layer can be evaluated as a causal convolution:

$$\mathbf{y}_t = \mathbf{D}\mathbf{u}_t + \sum_{i=0}^{t-1} \mathbf{K}_i\mathbf{u}_{t-i}.$$

This is convolution mode. It exposes parallelism over sequence positions. The layer can be trained like a convolutional sequence model, even though it also has a recurrent interpretation.

The central computational challenge is producing or applying the kernel efficiently. A generic dense $\bar{\mathbf{A}}$ makes $\bar{\mathbf{A}}^i$ expensive for long sequences and large state sizes. S4 and S5 are largely about choosing structures for $\bar{\mathbf{A}}$ that make this tractable.

3.9 The Role Of The Direct Term

The matrix $\mathbf{D}$ gives a direct path from current input to current output:

$$\mathbf{y}_t = \mathbf{C}\mathbf{x}_t + \mathbf{D}\mathbf{u}_t.$$

In convolution language, this is an instantaneous kernel component. Depending on indexing convention, it may be combined with $\mathbf{K}_0$ or treated separately.

The direct term is useful because the state path may be best at modeling temporal context, while the direct term preserves immediate information. Many deep architectures also have residual connections around the whole SSM block, giving another direct route for current features.

3.10 SISO And MIMO

A single-input single-output SSM has

$$\mathbf{u}_t \in \mathbb{R}, \quad \mathbf{y}_t \in \mathbb{R}.$$

Then $\mathbf{B}$ is a column vector, $\mathbf{C}$ is a row vector, and $\mathbf{K}_i$ is scalar.

A multi-input multi-output SSM has

$$\mathbf{u}_t \in \mathbb{R}^{C_{\text{in}}}, \quad \mathbf{y}_t \in \mathbb{R}^{C_{\text{out}}},$$

and $\mathbf{K}_i$ is a matrix:

$$\mathbf{K}_i \in \mathbb{R}^{C_{\text{out}} \times C_{\text{in}}}.$$

This is the natural form for modern sequence models, where every time step has many features.

S4 often uses SISO SSMs in parallel across channels, followed by separate channel-mixing layers. S5 emphasizes MIMO SSMs more directly. This is one of the practical simplifications that makes S5 easier to reason about as a neural network layer.

3.11 State Size, Model Width, And Channels

SSM notation can be confusing because there are several dimensions:

L	sequence length,
C_{in}	input channel count,
C_{out}	output channel count,
H	model width in a deep network,
N	SSM state size.

The state size N controls the dimension of the latent dynamical memory. The model width H is the feature width passed between blocks. In many architectures, $C_{\text{in}} = C_{\text{out}} = H$ inside the backbone, but N may be different.

This distinction matters for capacity. Increasing H gives the network more feature channels. Increasing N gives each SSM more dynamical memory. The two changes are not equivalent.

3.12 A Deep SSM Layer Is Not Just The SSM

The bare SSM recurrence is linear. A practical deep SSM block usually wraps it with other operations:

$\mathbf{z} \rightarrow \text{normalization} \rightarrow \text{input projection} \rightarrow \text{SSM temporal mixing} \rightarrow \text{nonlinearity or gate} \rightarrow \text{output projection} \rightarrow \text{residual}$

Different papers choose different wrappers, but this pattern is the right mental model. The SSM supplies long-range temporal mixing. The surrounding neural network supplies feature mixing, nonlinearity, and optimization stability.

This also explains why a linear SSM can be useful in a nonlinear model. A transformer attention operation is not the whole transformer block either. It is one structured mixing operation inside a larger residual architecture.

3.13 Why Structure Is Needed

The formula

$$\mathbf{K}_i = \mathbf{C} \bar{\mathbf{A}}^i \bar{\mathbf{B}}$$

looks simple, but a naive implementation has problems.

Computing long kernels can be expensive. If $\bar{\mathbf{A}}$ is dense, repeatedly multiplying by $\bar{\mathbf{A}}$ across a long sequence can be costly.

Training dense dynamics can be unstable. The eigenvalues of $\bar{\mathbf{A}}$ control memory. Bad parameterization can lead to exploding, vanishing, or poorly conditioned dynamics.

Long memory needs good initialization. Random matrices do not automatically produce a useful spectrum of timescales. For long sequences, the model needs modes that cover both short and long temporal dependencies.

Parallelism requires exploitable algebra. The convolution view is only computationally valuable if the kernel can be generated or applied efficiently. S4 uses special matrix structure. S5 simplifies toward diagonal structure and parallel scan. Mamba uses selective scan rather than fixed convolution.

3.14 SSMs Versus RNNs

The recurrence

$$\mathbf{x}_t = \bar{\mathbf{A}}\mathbf{x}_{t-1} + \bar{\mathbf{B}}\mathbf{u}_t$$

resembles a vanilla RNN, but there are crucial differences.

A standard RNN has nonlinear state updates:

$$\mathbf{h}_t = \tanh(\mathbf{W}_h\mathbf{h}_{t-1} + \mathbf{W}_u\mathbf{u}_t).$$

This nonlinearity makes the recurrence expressive, but it also prevents a simple convolutional representation.

A linear time-invariant SSM has a linear state update. That sounds less expressive, but it allows the recurrent and convolutional views to coexist:

$$\text{linear recurrence} \iff \text{causal convolution}.$$

The nonlinearity is moved outside the temporal recurrence and placed in the surrounding deep block. This design is one of the reasons SSM layers can train efficiently on long sequences.

3.15 SSMs Versus CNNs

An ordinary causal CNN learns kernel slices directly:

$$\mathbf{K}_0, \dots, \mathbf{K}_{K-1}.$$

An SSM generates them:

$$\mathbf{K}_i = \mathbf{C}\bar{\mathbf{A}}^i\bar{\mathbf{B}}.$$

This gives the SSM a compact way to represent long filters. The price is that the kernel must have the structure induced by the state dynamics.

So the comparison is:

CNN	free finite kernel	simple, local unless very large,
SSM	structured long kernel	compact, dynamical, stability-sensitive.

This is the practical reason to study SSMs after CNNs.

3.16 SSMs Versus Transformers

Transformers use content-dependent pairwise interactions:

$$\text{attention weight from } t \text{ to } s = \text{softmax}(q_t^\top k_s).$$

This gives direct access from one position to another, and the access pattern changes with the input content. That flexibility is powerful.

Linear time-invariant SSMs do not have this kind of content-dependent routing. The same kernel is applied regardless of the input values. Their advantage is different: they can model long temporal filters with linear scaling and recurrent online operation.

Mamba can be understood as an attempt to recover some of the content-dependent behavior of attention while keeping the linear-time recurrent structure of SSMs. It does this by making some SSM parameters input-dependent.

3.17 What To Remember

- A state space model separates input, latent state, and output.
- Continuous-time SSMs are discretized before being used on sampled sequences.
- Zero-order hold gives $\bar{A} = e^{\Delta A}$ and $\bar{B} = (\int_0^\Delta e^{\tau A} d\tau)B$.
- Continuous stability uses negative real eigenvalue parts; discrete stability uses eigenvalues inside the unit circle.
- A discretized linear time-invariant SSM can run in recurrent mode or convolution mode.
- The convolution kernel is $K_i = C\bar{A}^i\bar{B}$.
- S4 and S5 are mostly about making this kernel expressive, stable, and efficient.

3.18 Exercises

1. For the scalar continuous system $\dot{x}(t) = -2x(t)$, compute the discrete multiplier $\bar{a}$ when $\Delta = 0.1$.
2. If a continuous eigenvalue is $\lambda = -1 + 3i$, what is the magnitude of the corresponding discrete eigenvalue after step size Δ ?
3. Explain why zero-order hold is a reasonable assumption for binned neural data.
4. In your own words, distinguish state size N from model width H .
5. Starting from (3.2), unroll the recurrence for three steps and identify K_0, K_1, K_2 .

3.19 Prepares You For

The next chapter explains why modern SSMs needed special initializations and structured state matrices to store long histories without becoming unstable or computationally expensive. That brings us to HiPPO: a principled way to initialize recurrent memory for long sequence modeling.

Chapter 4

HiPPO And Memory

4.1 Why HiPPO Appears In SSM Papers

S4 did not begin from a random recurrent matrix. A central part of its success came from initializing the state dynamics with matrices derived from HiPPO: High-order Polynomial Projection Operators (Gu et al., 2020). HiPPO is a theory of online memory. It asks:

How can a finite-dimensional state store useful information about an entire past signal?

This is exactly the question long-sequence models face. A neural time series may contain relevant context spread across hundreds or thousands of time steps. A state vector cannot store the raw past exactly unless it grows with sequence length, so it must store a compressed summary. HiPPO gives a principled way to define such summaries.

The goal of this chapter is not to reproduce every HiPPO derivation. The original paper develops a general framework involving measures, orthogonal polynomials, and projection dynamics. For understanding S4 and S5, the most important idea is simpler:

HiPPO initializes a recurrent state to track polynomial coefficients of the past.

Those coefficients form a compact approximation of the input history. The resulting state matrices have a spectrum of timescales that is much better suited to long memory than a generic random matrix.

4.2 The Online Memory Problem

Suppose a scalar signal $u(t)$ arrives over time. At time t , the full history is the function

$$u_{\leq t} : \tau \mapsto u(\tau), \quad 0 \leq \tau \leq t.$$

Storing this full function is impossible with a fixed-size vector. Instead, we maintain a state

$$\mathbf{x}(t) \in \mathbb{R}^N$$

that summarizes the history. The update should be online: when a new input arrives, we update $\mathbf{x}(t)$ without replaying the whole past.

This is the same abstract problem solved by an RNN hidden state. The difference is that HiPPO specifies what the state should mean. Rather than letting a random recurrent matrix learn

memory from scratch, HiPPO defines $\mathbf{x}(t)$ as coefficients of a projection of the past signal onto a chosen basis.

The compression problem can be written informally as

$$u_{\leq t}(\tau) \approx \sum_{n=0}^{N-1} x_n(t) p_n(\tau; t),$$

where p_n are basis functions over the past interval and $x_n(t)$ are the coefficients stored in the state.

4.3 Projection Onto A Basis

A basis lets us approximate a function by a finite list of coefficients. In ordinary Euclidean geometry, projecting a vector onto an orthonormal basis gives coordinates:

$$\mathbf{v} \approx \sum_{n=0}^{N-1} c_n \mathbf{e}_n.$$

For functions, the same idea uses inner products:

$$c_n = \langle f, p_n \rangle.$$

If the basis functions p_n are orthonormal under the chosen inner product, then the coefficients c_n give the best approximation to f in the span of the first N basis functions.

HiPPO applies this idea to the input history. At time t , the state stores coefficients of a projection of the past signal. The basis is usually built from orthogonal polynomials, especially Legendre polynomials.

4.4 A Quick Reminder On Orthogonality

Two vectors are orthogonal if their dot product is zero. Two functions are orthogonal if their inner product is zero:

$$\langle f, g \rangle = \int f(\tau) g(\tau) w(\tau) d\tau.$$

The weight function $w(\tau)$ and interval of integration define the geometry. Different choices produce different families of orthogonal polynomials.

Legendre polynomials are orthogonal on an interval with a uniform weight. The first few on $[-1, 1]$ are

$$P_0(x) = 1, \quad P_1(x) = x, \quad P_2(x) = \frac{1}{2}(3x^2 - 1).$$

Higher-order polynomials capture finer variation. If we project a signal onto the first N Legendre polynomials, low-order coefficients capture coarse trends and high-order coefficients capture more detailed shape.

This gives a useful mental picture of HiPPO memory:

$$\mathbf{x}(t) = \text{coefficients of a polynomial approximation to recent history.}$$

4.5 Why The Basis Changes Over Time

The online memory problem is harder than a fixed projection problem because the interval of history changes as time advances. At time t , the past is $[0, t]$. At time $t + \Delta$, the past is $[0, t + \Delta]$. If we care more about recent history, the effective window or weighting may also shift over time.

HiPPO handles this by deriving differential equations for how the projection coefficients should change as new data arrive and as the underlying measure over history evolves. Those coefficient dynamics have the state space form

$$\frac{d}{dt}\mathbf{x}(t) = \mathbf{A}(t)\mathbf{x}(t) + \mathbf{B}(t)u(t),$$

with particular matrices determined by the polynomial family and measure.

For the most important variants used in SSM initialization, this can be turned into time-invariant or nearly time-invariant dynamics after a change of variables. That is what makes the connection to S4 possible.

4.6 Legendre Memory Intuition

The Legendre version of HiPPO is easiest to understand geometrically. Imagine always keeping a polynomial approximation to the recent past. The constant coefficient stores something like the average level. The linear coefficient stores something like the trend. Quadratic and higher coefficients store curvature and finer temporal structure.

As a new input arrives, all coefficients must update:

- the newest value must be inserted into the memory;
- older information must be shifted or rescaled because the time interval has changed;
- the polynomial approximation must remain the best projection under the chosen measure.

This produces a structured recurrent update. It is not arbitrary recurrence. It is the update required to keep the polynomial approximation correct as time moves forward.

4.7 Why This Helps Long Memory

Random recurrent matrices do not automatically preserve useful information over long horizons. They may forget too quickly, explode, or represent the past in a poorly conditioned way. HiPPO gives a deliberately constructed memory system.

The benefit is not that polynomial projection is the only good way to remember. The benefit is that it gives the model a strong starting point:

state dimensions correspond to organized timescales and basis components.

When used as an initialization for a learnable SSM, the model does not have to discover long memory from scratch. It starts from dynamics that already know how to compress history.

For long-range sequence modeling, this matters. A model that begins with only short timescale modes may never learn long dependencies well. HiPPO-derived matrices provide a spectrum of modes that can represent both coarse and fine structure over long contexts.

4.8 HiPPO As An SSM Initialization

The key output of HiPPO for S4 is a state matrix $\mathbf{A}$ and input matrix $\mathbf{B}$. These matrices define continuous-time dynamics:

$$\frac{d}{dt}\mathbf{x}(t) = \mathbf{A}\mathbf{x}(t) + \mathbf{B}u(t).$$

The state $\mathbf{x}(t)$ has an interpretation as projection coefficients. Then, as in Chapter 3, the continuous system can be discretized and used as a sequence layer.

In a deep SSM, the HiPPO matrices are not necessarily frozen. They are usually used to initialize learnable parameters. Training can adapt them to the task. This is analogous to initializing a convolutional filter bank with a useful signal-processing transform, then allowing gradient descent to tune it.

S4 uses a particular HiPPO-inspired matrix as the starting point, then applies additional structure so the resulting long convolution kernel can be computed efficiently. S5 keeps the broad idea of S4-style initialization but simplifies the state parameterization.

4.9 The HiPPO Matrix Shape

The exact entries of the HiPPO matrix are not essential on a first pass, but it is useful to know what kind of object it is. For the Legendre translated variant often discussed around S4, $\mathbf{A}$ is a structured lower-triangular matrix with entries determined by the polynomial orders. It is not a generic learned dense matrix.

At a high level:

- lower-order state coordinates track coarse components of the past;
- higher-order coordinates track finer components;
- the matrix couples these coordinates in the way required by online projection;
- the resulting dynamics have stable memory behavior across many timescales.

The original HiPPO paper defines several variants. They differ in the measure used to weight the past. Some variants emphasize the entire history; others emphasize recent history. For S4, the important fact is that HiPPO supplies a principled state matrix whose memory properties are far better than an arbitrary initialization.

4.10 Measures And Weighting The Past

The word “measure” in HiPPO can sound abstract, but the modeling idea is intuitive. When summarizing the past, we must decide which parts matter most. A uniform measure over the whole past treats all times equally. A recency-weighted measure gives more importance to recent inputs. A sliding-window-like measure focuses on a finite recent interval.

Different measures lead to different projection dynamics. This affects what the memory state represents:

whole-history weighting	→	global summary of all previous input,
recency weighting	→	stronger memory of recent input,
windowed weighting	→	compressed representation of a recent interval.

For deep learning, the exact measure is less important than the fact that HiPPO turns a choice about memory into concrete recurrent matrices.

4.11 What HiPPO Does Not Solve

HiPPO is not a full sequence model by itself. It solves an initialization and memory representation problem. Several other problems remain:

- How do we compute the resulting long convolution kernel efficiently?
- How do we parameterize the state matrix so it remains stable during training?
- How do we mix many input and output channels?
- How do we add nonlinear feature transformations around the linear memory?
- How do we scale the method on GPUs?

S4 answers these questions with structured state matrices and efficient kernel computation. S5 answers them with a simpler diagonal/MIMO formulation and parallel scan. Mamba changes the problem again by making the dynamics input-dependent.

4.12 Connection To Neural Time Series

For Utah array data, the signal is continuous in time but observed in bins. Speech motor cortex activity has temporal structure across multiple scales: fast changes around movement onset, slower context across phonemes or syllables, and session-level nuisance variation. A useful encoder needs memory, but not just one memory timescale.

HiPPO-style initialization is attractive because it gives an SSM a structured set of memory modes before task-specific training. In principle, some modes can track rapid local changes while others retain slower context. Whether this helps a particular SSL objective is empirical, but the architectural motivation is clear.

This also explains why SSMs are not merely “RNNs with different branding.” The point is not just to use recurrence. The point is to use a recurrence whose dynamics are designed to represent long histories and whose structure can be exploited computationally.

4.13 A Minimal Worked Picture

Suppose we approximate the recent past using three coefficients:

$$u_{\leq t}(\tau) \approx x_0(t)P_0(\tau) + x_1(t)P_1(\tau) + x_2(t)P_2(\tau).$$

Very roughly:

- $x_0(t)$ captures average level;
- $x_1(t)$ captures trend;
- $x_2(t)$ captures curvature.

When a new input arrives, the coefficients change. The update is not simply “replace the oldest value.” It must adjust the whole polynomial approximation. HiPPO derives the continuous-time update that performs this coefficient maintenance online.

Increasing the state size N adds more basis functions and therefore more detailed memory. In an SSM layer, N controls how rich this latent memory can be.

4.14 What To Remember

- HiPPO is a framework for online compression of a signal history.
- The state stores coefficients of a projection of the past onto basis functions.
- Orthogonal polynomial bases, especially Legendre bases, give organized memory coordinates.
- HiPPO-derived matrices initialize SSMs with useful long-memory dynamics.
- The exact HiPPO derivation is less important at first than the memory principle: maintain a finite-dimensional approximation of the past as new inputs arrive.
- S4 builds on HiPPO but also needs additional structure for efficient long-kernel computation.

4.15 Exercises

1. In your own words, explain the difference between storing the raw past and storing projection coefficients of the past.
2. Why might a polynomial basis be useful for compressing a temporal signal?
3. What kind of information would you expect low-order polynomial coefficients to store? What about high-order coefficients?
4. Explain why a good recurrent initialization matters for long-context modeling.
5. HiPPO gives a useful memory state, but not a complete deep sequence model. List two additional ingredients needed to turn it into an S4-style layer.

4.16 Prepares You For

S4 uses structured versions of HiPPO-inspired state matrices to get both long memory and efficient computation. The next chapter explains how S4 turns the SSM kernel

$$K_i = C \bar{A}^i \bar{B}$$

into a practical deep-learning layer for long sequences.

Chapter 5

S4

5.1 What Problem Did S4 Solve?

S4, the Structured State Space sequence model, made state space layers competitive for long-range deep sequence modeling (Gu et al., 2022). Its contribution was not simply “use an SSM.” The basic SSM equations are old. The contribution was to combine:

- HiPPO-inspired long-memory initialization;
- a structured parameterization of the state matrix;
- efficient computation of long convolution kernels;
- a deep residual neural-network block around the SSM layer.

The central computational goal is familiar from previous chapters. Given a discretized SSM,

$$\mathbf{x}_t = \bar{\mathbf{A}}\mathbf{x}_{t-1} + \bar{\mathbf{B}}\mathbf{u}_t, \quad \mathbf{y}_t = \mathbf{C}\mathbf{x}_t + \mathbf{D}\mathbf{u}_t,$$

we want to apply the convolution kernel

$$\mathbf{K}_i = \mathbf{C}\bar{\mathbf{A}}^i\bar{\mathbf{B}}$$

over very long sequences. S4 asks: how can this be expressive, stable, and fast?

5.2 Why Ordinary SSMs Are Not Enough

A generic dense state matrix $\mathbf{A} \in \mathbb{R}^{N \times N}$ creates several problems.

Kernel generation is expensive. Computing $\mathbf{C}\bar{\mathbf{A}}^i\bar{\mathbf{B}}$ for many lags i can require repeated dense matrix multiplications. For long sequences, this is too slow.

Training can be unstable. The eigenvalues of $\bar{\mathbf{A}}$ control the memory. If they drift outside the unit circle, the recurrence can explode. If they move too far inside, memory vanishes.

Random initialization is poor for long memory. Long-context tasks need a useful range of timescales from the start. HiPPO helps, but the raw HiPPO matrices also need to be represented in a computationally tractable way.

Naive recurrence loses parallelism. Running the recurrence step by step is natural for inference, but it gives up the training parallelism that makes convolutions and transformers attractive.

S4 is best understood as an answer to these four problems at once.

5.3 The S4 Layer At A High Level

An S4 layer maps an input sequence to an output sequence by:

1. parameterizing a continuous-time SSM;
2. discretizing it using a step size Δ ;
3. generating a long causal convolution kernel from the discretized SSM;
4. applying that kernel to the sequence;
5. adding nonlinearities, channel mixing, and residual connections around it.

At the temporal-mixing core, S4 is still an LTI system:

$$\mathbf{y}_t = \sum_{i=0}^{t-1} \mathbf{K}_i \mathbf{u}_{t-i}.$$

The novelty is how $\mathbf{K}_i$ is parameterized and computed.

5.4 SISO SSMs In S4

The simplest S4 presentation uses single-input single-output SSMs. For one channel:

$$u_t \in \mathbb{R}, \quad y_t \in \mathbb{R},$$

and

$$\mathbf{x}_t \in \mathbb{R}^N.$$

The kernel is scalar:

$$K_i = \mathbf{C} \bar{\mathbf{A}}^i \bar{\mathbf{B}}.$$

For a multi-channel neural network, S4 can use separate SISO SSMs across channels, with pointwise projections or other mixing layers before and after the SSM operation. So one should not picture the whole network as scalar. Rather, the SSM temporal filter itself is often channelwise, while channel interactions are handled by surrounding linear layers.

This design choice will matter when we compare S4 with S5. S5 moves toward a more direct multi-input multi-output SSM formulation.

5.5 HiPPO Initialization In S4

S4 begins from a HiPPO-inspired continuous-time matrix. From the previous chapter, the purpose of HiPPO is to initialize a state that can compress long histories using polynomial projection dynamics. This gives the SSM a good memory structure before training.

The rough pipeline is:

HiPPO continuous dynamics $\rightarrow$ structured SSM parameterization $\rightarrow$ discretized convolution kernel.

This is important because S4’s long-memory behavior is not just a consequence of using a large state. It comes from initializing and parameterizing the state matrix so that it has useful modes across many timescales.

5.6 Normal Matrices

To understand S4’s structure, first recall diagonalization. If

$$\mathbf{A} = \mathbf{V}\mathbf{\Lambda}\mathbf{V}^{-1},$$

then powers of $\mathbf{A}$ are easy in the eigenbasis:

$$\mathbf{A}^i = \mathbf{V}\mathbf{\Lambda}^i\mathbf{V}^{-1}.$$

The problem is that an arbitrary diagonalization can be numerically poorly conditioned. If $\mathbf{V}^{-1}$ is unstable, the computation may be fragile.

A normal matrix is a complex matrix satisfying

$$\mathbf{A}^* \mathbf{A} = \mathbf{A} \mathbf{A}^*,$$

where $\mathbf{A}^*$ is the conjugate transpose. Normal matrices are especially nice because they can be diagonalized by a unitary matrix:

$$\mathbf{A} = \mathbf{V}\mathbf{\Lambda}\mathbf{V}^*.$$

Unitary matrices preserve norms, which makes this representation much better conditioned.

Diagonal matrices are the simplest normal matrices. Symmetric real matrices and Hermitian complex matrices are also normal. S4 exploits the fact that certain HiPPO-related matrices can be represented as a normal part plus a low-rank correction.

5.7 Normal Plus Low-Rank Structure

S4 uses a structured state matrix often described as normal plus low-rank, or NPLR:

$$\mathbf{A} = \mathbf{A}_{\text{normal}} - \mathbf{P}\mathbf{Q}^*,$$

where $\mathbf{A}_{\text{normal}}$ is normal and $\mathbf{P}\mathbf{Q}^*$ is low-rank. The exact signs and forms depend on the convention, but the idea is:

HiPPO-like matrix = diagonalizable normal matrix + small low-rank correction.

This is the central S4 algebraic trick. The normal part gives efficient diagonalization. The low-rank correction preserves the useful HiPPO memory structure. Together, they make it possible to compute long convolution kernels efficiently without throwing away the long-memory initialization.

On a first reading, you do not need to derive the NPLR decomposition. What matters is the modeling role:

- diagonal or normal dynamics are computationally convenient;
- raw HiPPO dynamics are memory-useful but not directly convenient;
- NPLR structure lets S4 keep both advantages.

5.8 Complex Diagonal Dynamics

After transforming into a convenient basis, S4 works with complex-valued diagonal or almost-diagonal dynamics. A diagonal state matrix has entries

$$\lambda_1, \dots, \lambda_N.$$

The kernel contains terms like

$$\lambda_j^i.$$

As Chapter 2 discussed, complex eigenvalues represent decaying oscillatory modes:

$$\lambda_j = r_j e^{i\omega_j} \Rightarrow \lambda_j^i = r_j^i e^{i\omega_j i}.$$

This makes S4 filters rich. A long kernel can mix many decays and oscillations. In signal terms, the layer can learn filters with a broad range of temporal and frequency responses.

The model output remains real-valued. Complex arithmetic is a convenient internal parameterization, not a claim that the data are complex.

5.9 Generating The Convolution Kernel

For a sequence of length L , S4 needs the kernel

$$K_0, K_1, \dots, K_{L-1}.$$

In a scalar SISO case,

$$K_i = \mathbf{C} \bar{\mathbf{A}}^i \bar{\mathbf{B}}.$$

If $\bar{\mathbf{A}}$ is diagonal,

$$\bar{\mathbf{A}} = \text{diag}(\bar{\lambda}_1, \dots, \bar{\lambda}_N),$$

then

$$K_i = \sum_{j=1}^N C_j B_j \bar{\lambda}_j^i.$$

So the kernel is a sum of exponentials. This is already much easier than powering a dense matrix.

S4's full NPLR case is more complicated, but the same broad idea remains: exploit the structured state matrix to compute the long kernel quickly.

5.10 The Cauchy Kernel Idea

The S4 paper emphasizes efficient computation through Cauchy kernels. The details are technical, but the conceptual path is manageable.

Rather than compute every time-domain coefficient K_i by repeated powers, one can work with the generating function or frequency-domain representation of the SSM. For diagonal dynamics, terms of the form

$$\frac{1}{\omega - \lambda_j}$$

appear when evaluating the transfer function at frequency-like points ω . A matrix with entries of this reciprocal-difference form is called a Cauchy matrix.

So the high-level statement is:

SSM transfer-function evaluation $\rightsquigarrow$ structured Cauchy matrix computation.

Efficient Cauchy multiplication lets S4 generate the convolution kernel or its frequency representation much faster than a naive dense approach. For learning S4 conceptually, it is enough to know that this is the specialized numerical routine that makes the structured SSM practical at long sequence lengths.

5.11 Convolutional Training

During training, S4 usually uses convolution mode. For an input sequence $u_{1:L}$, it computes a kernel $K_{0:L-1}$ and applies

$$y_t = \sum_{i=0}^{t-1} K_i u_{t-i}.$$

This can be done in parallel over the sequence, often using FFT-based convolution or related efficient methods.

This is a major advantage over a naive RNN-style recurrence. The underlying model has a state-space interpretation, but training does not have to process one time step after another.

The computational pattern is:

generate long kernel $\rightarrow$ apply causal convolution $\rightarrow$ use ordinary backpropagation.

5.12 Recurrent Inference

The same S4 layer can also be run recurrently:

$$\mathbf{x}_t = \bar{\mathbf{A}}\mathbf{x}_{t-1} + \bar{\mathbf{B}}u_t, \quad y_t = \mathbf{C}\mathbf{x}_t + \mathbf{D}u_t.$$

This is useful for online inference or autoregressive generation. Instead of storing the whole past sequence, the model stores the current state $\mathbf{x}_t$.

This duality is one of the most attractive features of LTI SSMs:

parallel convolution for training and recurrent state update for deployment.

Transformers have a different deployment story: causal attention can cache previous keys and values, but memory grows with context length. An SSM recurrent state has fixed size.

5.13 Discretization In S4

S4 parameterizes continuous-time dynamics and then discretizes them. As in Chapter 3, discretization depends on a step size Δ :

$$\bar{\mathbf{A}} = e^{\Delta \mathbf{A}}, \quad \bar{\mathbf{B}} = \left(\int_0^\Delta e^{\tau \mathbf{A}} d\tau \right) \mathbf{B}.$$

The step size can be learned. This gives the layer control over effective timescales. Different channels can learn different discretizations, allowing the network to adapt the continuous dynamics to the sampled sequence.

For practical understanding, remember:

Δ controls how continuous memory modes appear at discrete sequence steps.

5.14 The S4 Block

The SSM kernel is only the temporal core. A deep S4 model wraps this core in a residual block. A simplified block is:

$z \rightarrow \text{normalization} \rightarrow \text{linear projection} \rightarrow \text{S4 temporal convolution} \rightarrow \text{activation and dropout} \rightarrow \text{linear projection}$

Exact details vary across implementations. Some versions use gating, different normalization placements, or additional feed-forward layers. The important design is the same as in transformers: the specialized sequence operation is one component inside a deep residual network.

This matters because the bare SSM is linear. Nonlinear modeling power comes from stacking blocks with activations, gates, projections, and residual paths.

5.15 What S4 Learns

S4 learns the parameters that determine the convolution kernel. Depending on the implementation, these include:

- eigenvalue-like parameters controlling decay and oscillation;
- input and output vectors analogous to $\mathbf{B}$ and $\mathbf{C}$;
- step sizes Δ ;
- direct terms and surrounding channel-mixing projections.

The learned kernel can be very long. But unlike a CNN kernel, it is not an arbitrary list of free parameters. It is generated by a structured dynamical system. That structure is the inductive bias that gives S4 its efficiency and long-memory behavior.

5.16 Why S4 Was Important

S4 showed that state space layers could compete with attention and other sequence models on difficult long-range benchmarks. The key lesson was not just empirical. It changed the architectural picture:

recurrence $\neq$ slow RNN training by necessity.

A carefully structured recurrent model can expose a convolutional form for training while retaining a recurrent form for inference. This is the foundation on which S5 and later models build.

5.17 Limitations And Complexity

S4 is powerful, but it is not simple. Its implementation involves complex-valued parameters, special matrix structure, discretization details, and specialized kernel computation. These details make it less straightforward than a standard convolution or vanilla RNN.

This complexity motivated simplified variants. S5 keeps the state-space approach but removes some of S4's harder machinery by using diagonal state matrices and a parallel scan formulation. Mamba later changes the emphasis again by adding input-dependent selectivity.

5.18 A Reading Strategy For The S4 Paper

When reading [Gu et al. \(2022\)](#), it helps to separate the paper into layers:

1. The modeling layer: an SSM defines a long convolution kernel.
2. The memory layer: HiPPO gives a strong long-memory initialization.
3. The algebra layer: NPLR structure makes the matrix tractable.
4. The numerical layer: Cauchy kernel computation makes long convolution efficient.
5. The neural-network layer: S4 is placed inside deep residual blocks.

Trying to understand all five layers at once is difficult. The paper becomes much easier once you can identify which layer a given equation belongs to.

5.19 What To Remember

- S4 is a structured state space layer for long sequence modeling.
- It uses HiPPO-inspired dynamics to initialize long memory.
- The SSM kernel is still $\mathbf{K}_i = \mathbf{C}\bar{\mathbf{A}}^i\bar{\mathbf{B}}$.
- S4 uses normal-plus-low-rank structure to keep HiPPO-like dynamics tractable.
- Complex modes represent decays and oscillations.
- Training can use convolution mode; online inference can use recurrent mode.
- The S4 layer is embedded inside a nonlinear residual block.
- S4's power comes with implementation complexity, motivating S5.

5.20 Exercises

1. Explain why a generic dense $\mathbf{A}$ is problematic for long-sequence SSMs.
2. In your own words, what does NPLR structure buy us?
3. Why are complex eigenvalues natural for temporal filters?
4. Describe the difference between S4's convolutional training mode and recurrent inference mode.
5. When reading the S4 paper, identify whether a difficult equation belongs mainly to memory initialization, matrix structure, numerical kernel computation, or the neural block design.

5.21 Prepares You For

S5 keeps the state-space viewpoint but simplifies the parameterization and changes the multi-channel treatment. The next chapter explains how S5 moves from S4's more elaborate NPLR machinery toward diagonal state matrices, MIMO SSMs, and parallel scan.

Chapter 6

S5

6.1 Why S5 Exists

S4 showed that structured state space layers could model long sequences effectively, but it came with significant machinery: normal-plus-low-rank matrices, complex kernel generation, Cauchy computations, and a fairly intricate implementation. S5, short for Simplified State Space Layers, keeps the core SSM idea while simplifying the architecture and computation ([Smith et al., 2023](#)).

At a high level, S5 makes three important moves:

- use diagonal state matrices rather than S4’s more elaborate NPLR form;
- use multi-input multi-output SSMs rather than many independent SISO filters;
- use parallel scan to evaluate the recurrence efficiently.

This makes S5 a particularly good model to understand after S4. It preserves the state-space perspective, but many pieces become easier to reason about.

6.2 The S5 Recurrence

The discrete SSM recurrence is still the starting point:

$$\mathbf{x}_t = \bar{\mathbf{A}}\mathbf{x}_{t-1} + \bar{\mathbf{B}}\mathbf{u}_t, \quad \mathbf{y}_t = \mathbf{C}\mathbf{x}_t + \mathbf{D}\mathbf{u}_t.$$

S5 uses a diagonal state transition:

$$\bar{\mathbf{A}} = \text{diag}(\bar{\lambda}_1, \dots, \bar{\lambda}_N).$$

Then each state coordinate evolves independently before the readout:

$$x_t[j] = \bar{\lambda}_j x_{t-1}[j] + (\bar{\mathbf{B}}\mathbf{u}_t)[j].$$

The coordinates interact through $\bar{\mathbf{B}}$ and $\mathbf{C}$, not through a dense transition matrix. This is much simpler than a generic dense $\bar{\mathbf{A}}$.

The kernel is still

$$\mathbf{K}_i = \mathbf{C}\bar{\mathbf{A}}^i\bar{\mathbf{B}}.$$

With diagonal $\bar{\mathbf{A}}$,

$$\bar{\mathbf{A}}^i = \text{diag}(\bar{\lambda}_1^i, \dots, \bar{\lambda}_N^i),$$

so the kernel is a mixture of independent exponential modes.

6.3 Diagonal Does Not Mean Trivial

It is tempting to think a diagonal state matrix is too simple. But diagonal dynamics can still be expressive when combined with input and output mixing.

The input matrix $\bar{\mathbf{B}}$ maps the input vector into all state coordinates:

$$\bar{\mathbf{B}}\mathbf{u}_t \in \mathbb{C}^N.$$

The output matrix $\mathbf{C}$ mixes all state coordinates back into output channels:

$$\mathbf{C}\mathbf{x}_t \in \mathbb{R}^{C_{\text{out}}}.$$

So each output channel can depend on many state modes, and each input channel can excite many state modes. The state modes themselves are independent, but the layer as a whole can still mix channels richly.

This is the same principle that makes diagonalization useful in linear systems. In the right basis, dynamics are independent modes; input and output projections determine how those modes are used.

6.4 Complex Diagonal Modes

S5 commonly uses complex diagonal modes. Each eigenvalue-like parameter can be written as

$$\lambda_j = \alpha_j + i\omega_j.$$

In continuous time, stability requires

$$\text{Re}(\lambda_j) = \alpha_j < 0.$$

After discretization with step size Δ , the discrete eigenvalue is

$$\bar{\lambda}_j = e^{\Delta\lambda_j}.$$

Its magnitude is

$$|\bar{\lambda}_j| = e^{\Delta\alpha_j},$$

so negative real part gives a magnitude below 1. The imaginary part ω_j controls oscillation.

Thus, S5's diagonal state can represent a bank of decaying oscillatory filters:

$$\bar{\lambda}_j^t = e^{\Delta\alpha_j t} e^{i\Delta\omega_j t}.$$

This is a powerful basis for temporal modeling.

6.5 MIMO State Space Layers

One of S5's main changes from S4 is the move toward multi-input multi-output SSMs. Instead of treating channels as many independent SISO filters, S5 allows a single SSM layer to map multiple input channels to multiple output channels:

$$\mathbf{u}_t \in \mathbb{R}^H, \quad \mathbf{y}_t \in \mathbb{R}^H, \quad \mathbf{x}_t \in \mathbb{C}^N.$$

Here H is model width and N is state size.

The parameter shapes are

$$\bar{\mathbf{B}} \in \mathbb{C}^{N \times H}, \quad \mathbf{C} \in \mathbb{C}^{H \times N}.$$

The resulting kernel has shape

$$\mathbf{K}_i \in \mathbb{R}^{H \times H}$$

after taking the real-valued output convention. So at each lag i , the layer can map all input channels to all output channels.

This MIMO view is appealing because it makes the SSM layer feel more like a standard sequence block over model-width features rather than a collection of independent scalar filters plus separate mixing.

6.6 Discretization And Learned Timescales

Like S4, S5 can start from continuous-time parameters and discretize them. For diagonal $\mathbf{A}$, discretization is especially simple:

$$\bar{\lambda}_j = e^{\Delta_j \lambda_j}.$$

The step size Δ_j may be learned or parameterized. This gives each mode its own effective timescale.

The input matrix also needs discretization. For a scalar diagonal mode,

$$\bar{B}_j = \left(\int_0^{\Delta_j} e^{\tau \lambda_j} d\tau \right) B_j = \frac{e^{\Delta_j \lambda_j} - 1}{\lambda_j} B_j$$

when $\lambda_j \neq 0$. The exact implementation may use numerically stable variants, but the idea is the same as Chapter 3: continuous dynamics are converted into discrete sequence updates.

6.7 Parallel Scan

S5 emphasizes parallel scan as a way to evaluate the recurrence efficiently. To understand scan, start with a recurrence of the form

$$\mathbf{x}_t = \mathbf{A}_t \mathbf{x}_{t-1} + \mathbf{b}_t.$$

For S5 with fixed diagonal $\bar{\mathbf{A}}$,

$$\mathbf{A}_t = \bar{\mathbf{A}}, \quad \mathbf{b}_t = \bar{\mathbf{B}} \mathbf{u}_t.$$

Each time step defines an affine map:

$$F_t(\mathbf{x}) = \mathbf{A}_t \mathbf{x} + \mathbf{b}_t.$$

Composing two adjacent updates gives

$$F_2(F_1(\mathbf{x})) = \mathbf{A}_2 \mathbf{A}_1 \mathbf{x} + \mathbf{A}_2 \mathbf{b}_1 + \mathbf{b}_2.$$

So the composition of two affine maps is another affine map:

$$(\mathbf{A}_2, \mathbf{b}_2) \circ (\mathbf{A}_1, \mathbf{b}_1) = (\mathbf{A}_2 \mathbf{A}_1, \mathbf{A}_2 \mathbf{b}_1 + \mathbf{b}_2).$$

This composition is associative. Associativity means we can compute all prefix compositions with a parallel scan algorithm. That gives the same result as stepping through the recurrence, but with much more parallelism on suitable hardware.

This matters because it gives S5 another path to efficient training. Instead of explicitly forming a long convolution kernel, one can evaluate the recurrence through an associative parallel prefix operation.

6.8 Scan Versus Convolution

For a time-invariant linear SSM, recurrent mode and convolution mode are mathematically equivalent. S4 leans heavily on generating and applying the convolution kernel. S5’s parallel scan view emphasizes efficient recurrence evaluation.

The distinction is practical:

Convolution mode	:	construct or apply $\mathbf{K}_i = \mathbf{C}\bar{\mathbf{A}}^i\bar{\mathbf{B}}$,
Scan mode	:	parallelize the recurrence updates directly.

Both exploit structure. Convolution exploits time invariance and fast filtering. Scan exploits associativity of recurrence composition. Mamba will also rely heavily on scan, because its input-dependent dynamics break the fixed convolution kernel.

6.9 Initialization From S4 And HiPPO

S5 simplifies S4, but it does not discard the long-memory initialization story. It can use initializations derived from S4/HiPPO-style dynamics. The goal is still to start with a useful spectrum of modes rather than random short-memory behavior.

Conceptually:

HiPPO/S4 initialization $\rightarrow$ diagonalized modes $\rightarrow$ S5 diagonal SSM.

The practical result is that S5 begins with modes covering a range of timescales. Training then adapts those modes, along with $\bar{\mathbf{B}}$, $\mathbf{C}$, step sizes, and surrounding neural network parameters.

6.10 The S5 Block

A simplified S5 block has the same broad structure as other residual sequence blocks:

$\mathbf{z} \rightarrow$ normalization $\rightarrow$ linear projection $\rightarrow$ S5 temporal mixing $\rightarrow$ nonlinearity or gate $\rightarrow$ linear projection $\rightarrow$ resid

Implementation details vary, but this is the right conceptual structure.

The S5 temporal mixing step is the diagonal MIMO SSM. The rest of the block supplies feature mixing, nonlinearity, regularization, and stable deep optimization.

6.11 State Size And Model Width

S5 makes the distinction between state size and model width especially important.

H	number of feature channels passed between blocks,
N	number of SSM state modes.

Increasing H gives the network more representation channels. Increasing N gives the SSM more temporal modes. In a diagonal SSM, each state coordinate is one mode, so N is directly tied to the richness of the temporal filter bank.

For neural time series, this distinction is practically important. A model might need enough width to represent neural features and enough state size to capture dynamics across speech-relevant timescales. These are related but not identical forms of capacity.

6.12 S5 Compared With S4

The cleanest comparison is:

- S4 : NPLR structure, SISO emphasis, specialized kernel computation,
- S5 : diagonal structure, MIMO emphasis, parallel scan.

S4 is closer to the full structured HiPPO matrix story. S5 is simpler and often easier to implement and adapt. The simplification is not free: diagonal dynamics are a stronger restriction than NPLR dynamics. But the MIMO formulation and learnable projections can recover substantial expressivity in practice.

For many applied projects, S5 is attractive because it provides the core SSM advantages without requiring the full S4 implementation stack.

6.13 Why S5 Is Interesting For Neural Data

Neural time series are not token sequences in the language-modeling sense. They are continuous, noisy, dynamical signals. SSMs are a natural inductive bias because they model temporal evolution through latent state.

S5 is especially appealing for this setting because:

- it is causal by construction when run left to right;
- it can maintain fixed-size recurrent state for online decoding;
- diagonal modes provide interpretable timescale and oscillation structure;
- MIMO input/output maps can mix many neural channels;
- parallel scan makes training more practical than naive recurrence.

This does not guarantee good downstream decoding. The local project notes already show that SSL objective design, nuisance structure, normalization, and evaluation matter a great deal. But architecturally, S5 is a reasonable backbone to test for causal neural sequence modeling.

6.14 Limitations

S5 also has limitations.

Fixed dynamics. Like S4, a standard S5 layer is linear time invariant inside the temporal core. The same dynamics apply regardless of input content. This is efficient, but less flexible than attention’s content-dependent routing.

Diagonal state transition. Diagonal dynamics are simple and efficient, but they restrict interactions between state modes. Mode interaction must happen through input/output projections and nonlinear stacking across layers.

Objective dependence. A good SSM backbone does not by itself solve self-supervised learning. If an objective can be solved through local recording-context shortcuts or by predicting a trivial normalized mean, S5 may learn unhelpful representations.

6.15 A Reading Strategy For The S5 Paper

When reading [Smith et al. \(2023\)](#), separate the ideas into four layers:

1. The simplification layer: replace S4's harder structured matrices with diagonal state dynamics.
2. The channel layer: use MIMO SSMs for richer input-output mixing.
3. The algorithm layer: use parallel scan to evaluate recurrences efficiently.
4. The architecture layer: place S5 inside a deep residual sequence model.

If an equation feels hard, ask which of these layers it belongs to. That usually makes the paper easier to navigate.

6.16 What To Remember

- S5 is a simplified state space sequence layer built after S4.
- It uses diagonal state transitions, often with complex-valued modes.
- Diagonal modes represent decays and oscillations.
- MIMO $\bar{\mathbf{B}}$ and $\mathbf{C}$ matrices let the layer mix many input and output channels.
- Parallel scan evaluates recurrence updates efficiently using associativity.
- S5 preserves the recurrent/convolutional SSM spirit while reducing S4's implementation complexity.
- For neural sequence modeling, S5 is attractive because it is causal, efficient, and organized around latent dynamics.

6.17 Exercises

1. For diagonal $\bar{\mathbf{A}} = \text{diag}(\lambda_1, \lambda_2)$, write $\bar{\mathbf{A}}^i$ explicitly.
2. Explain why diagonal state dynamics can still produce rich outputs when paired with dense $\bar{\mathbf{B}}$ and $\mathbf{C}$ matrices.
3. Show that composing affine recurrence updates $(\mathbf{A}_2, \mathbf{b}_2) \circ (\mathbf{A}_1, \mathbf{b}_1)$ gives $(\mathbf{A}_2\mathbf{A}_1, \mathbf{A}_2\mathbf{b}_1 + \mathbf{b}_2)$.
4. In your own words, compare convolution mode and scan mode.
5. Why might fixed LTI dynamics be limiting for tasks that require content-dependent selection?

6.18 Prepares You For

Mamba modifies the SSM idea by making the dynamics selective, meaning some SSM parameters depend on the current input. This gives up the fixed convolution kernel but keeps an efficient scan-based recurrent computation.

Chapter 7

Mamba

Purpose

This chapter explains Mamba as a selective state space model: still linear-time and recurrent in spirit, but no longer a fixed linear time-invariant convolution.

Assumed Background

The reader understands SSMs, convolutional training for time-invariant systems, and the transformer idea of content-dependent routing through attention.

Key Concepts To Define

- Selection as input-dependent SSM parameters.
- Why fixed LTI convolution cannot implement content-based routing.
- Input-dependent $\mathbf{B}$, $\mathbf{C}$, and step size Δ .
- Why selectivity breaks ordinary convolution mode.
- Hardware-aware parallel scan.
- The Mamba block: projection, local convolution, selective scan, gating, and output projection.
- Differences between attention, S4/S5, and Mamba for long sequences.

Primary Sources

The main source is [Gu and Dao \(2023\)](#). The optional follow-up is Mamba-2 and structured state space duality ([Dao and Gu, 2024](#)), which can be introduced after the original Mamba chapter is solid.

Prepares You For

The final comparison chapter places Mamba alongside CNNs, RNNs, transformers, S4, and S5, with special attention to Utah array self-supervised learning.

Chapter 8

Comparison And Project Relevance

Purpose

This chapter turns the technical material into modeling judgment. The goal is to compare architecture families and make clear why S5 and Mamba are plausible backbones for self-supervised neural decoding.

Architectures To Compare

- CNNs: fixed finite filters with strong parallelism.
- RNNs: recurrent hidden states with simple online inference but limited parallelism.
- Transformers: content-based global mixing with high memory cost at long context.
- S4: structured long convolution kernels generated by SSM parameters.
- S5: simplified diagonal MIMO SSMs with efficient parallel scan.
- Mamba: selective SSMs with input-dependent dynamics and linear-time scaling.

Comparison Axes

- Parallel training and online inference.
- Long-range memory and stability.
- Content-dependent routing versus fixed dynamics.
- Parameter efficiency and state size.
- Causal operation and future leakage.
- Suitability for Utah array SSL objectives such as future prediction and masked reconstruction.

Project Connection

The local project has already identified S5 and Mamba as the main SSM-family candidates for causal self-supervised Utah array modeling. This chapter should connect the theory to practical choices: raw-bin versus causal-patch inputs, subject/session adaptation, frozen phoneme probes, and whether SSL metrics reflect speech-relevant structure rather than recording-context shortcuts.

Exit Criteria For The Notes

After finishing the full document, the reader should be able to read the S4, S5, and Mamba papers without getting stuck on core notation, understand the computational tradeoffs, and reason about which model family is appropriate for a causal neural decoder.

Bibliography

- Dao, T. and Gu, A. (2024). Transformers are SSMS: Generalized models and efficient algorithms through structured state space duality. *arXiv preprint arXiv:2405.21060*.
- Goodfellow, I., Bengio, Y., and Courville, A. (2016). *Deep Learning*. MIT Press.
- Gu, A. and Dao, T. (2023). Mamba: Linear-time sequence modeling with selective state spaces. *arXiv preprint arXiv:2312.00752*.
- Gu, A., Dao, T., Ermon, S., Rudra, A., and Ré, C. (2020). HiPPO: Recurrent memory with optimal polynomial projections. In *Advances in Neural Information Processing Systems*.
- Gu, A., Goel, K., and Ré, C. (2022). Efficiently modeling long sequences with structured state spaces. In *International Conference on Learning Representations*.
- Li, F.-F., Johnson, J., and Yeung, S. (2024). CS231n: Convolutional neural networks for visual recognition. Stanford University course notes.
- Smith, J. T. H., Warrington, A., and Linderman, S. W. (2023). Simplified state space layers for sequence modeling. In *International Conference on Learning Representations*.